Design and Implementation of a Cost-Based Relational Query Engine

Harshit Jaiswal (230460) Aamir Ahmad (230010)

Abstract

This report presents the complete design, implementation, and empirical evaluation of a cost-based relational query engine built from scratch in C++17 for the CS315 Database Systems course project.The system faithfully implements the full query processing pipeline found in industrial database management systems: SQL parsing, relational algebra translation, cost-based optimization, and pipelined physical execution.This report details the internal mechanics of every core file from the storage simulation to the dynamic programming core and the Volcano execution model. Furthermore, we provide a deep dive into the mathematical formulas driving the cost model, demonstrating how disk I/O constraints ($s \times t_s + b \times t_b$) force the optimizer to dynamically pivot between Block Nested-Loop, Hash, and Sort-Merge joins. Finally, an analysis of the automated testing suite validates the engine's capacity to adapt to changing schema metadata and massive data volumes in real-time.

1 Introduction

Modern database management systems (DBMS) rely on a pipelined architecture to convert declarative SQL queries into highly optimized, physically executable instructions. The complexity of this process lies in bridging the gap between theoretical relational algebra and the physical realities of disk hardware, where random-access seeks are orders of magnitude slower than sequential block transfers.

This project implements a relational query engine from scratch in C++. The system enforces a strict main-memory constraint of $M = 25$ blocks. This artificial constraint forces the engine to discard naive algorithms and instead utilize disk-aware algorithms like Recursive Partitioning Hash Join and External Sort-Merge Join, mirroring the behavior of enterprise databases dealing with terabytes of out-of-core data.

The purpose of this report is to dissect the engine's codebase and mathematical models. We provide a file-by-file analysis, explaining the purpose of each class, the mathematical derivations of our cost models, and how control flows through the system to execute complex, multi-table queries.

2 System Architecture and Data Flow

The query processing pipeline is strictly divided into three phases, ensuring that physical execution algorithms never handle raw string parsing, and the optimizer is never bogged down by actual data manipulation.

The standard data flow is as follows:

1. **Query Parsing:** A SQL string is submitted to the Hyrise `hsql` library, which produces an Abstract Syntax Tree (AST). The `SQLToRATreeConverter` traverses this AST and constructs a logical Relational Algebra (RA) tree of `TableNode`, `SelectNode`, `ProjectNode`, and `JoinNode` objects. In parallel, a `JoinGraph` is constructed, flattening the query into a list of base tables, join predicates, and selection conditions that the optimizer can directly enumerate. conditions that the optimizer can directly enumerate.

2. **Cost Optimization:** `Optimizer.h` applies a Selinger-style Dynamic Programming algorithm, evaluating all 2^n subsets of the n base tables via bitmask enumeration. The search space is restricted to left-deep join trees. For each candidate join, `CostModel.h` prices all three physical algorithms (BNLJ, Hash Join, Sort-Merge Join) using simulated disk I/O costs. The result is an optimal `PlanNode` tree carrying both the chosen join order and the selected physical algorithm at each node.

3. **Execution:** The `PlanNode` tree is handed to `ExecutionEngine.h`, which recursively instantiates a corresponding tree of `Operator` objects following the Volcano Iterator Model. Each operator exposes an `open()` / `next()` / `close()` interface, pulling rows on demand from the `InMemoryDatabase` and passing them up the pipeline with minimal intermediate materialization.

3 Mathematical Cost Modeling

The mathematical core of the query planner resides in `CostModel.h`. The rigorous interconnectivity between the Optimizer and the Cost Model is what separates this engine from trivial heuristic models. By enforcing I/O constraints, the engine respects the physics of magnetic hard drives.

3.1 The I/O Cost Formula

All algorithms are evaluated based on their total simulated I/O time. Let b be the number of block transfers (sequential reads/writes) and s be the number of random-access disk seeks. Let

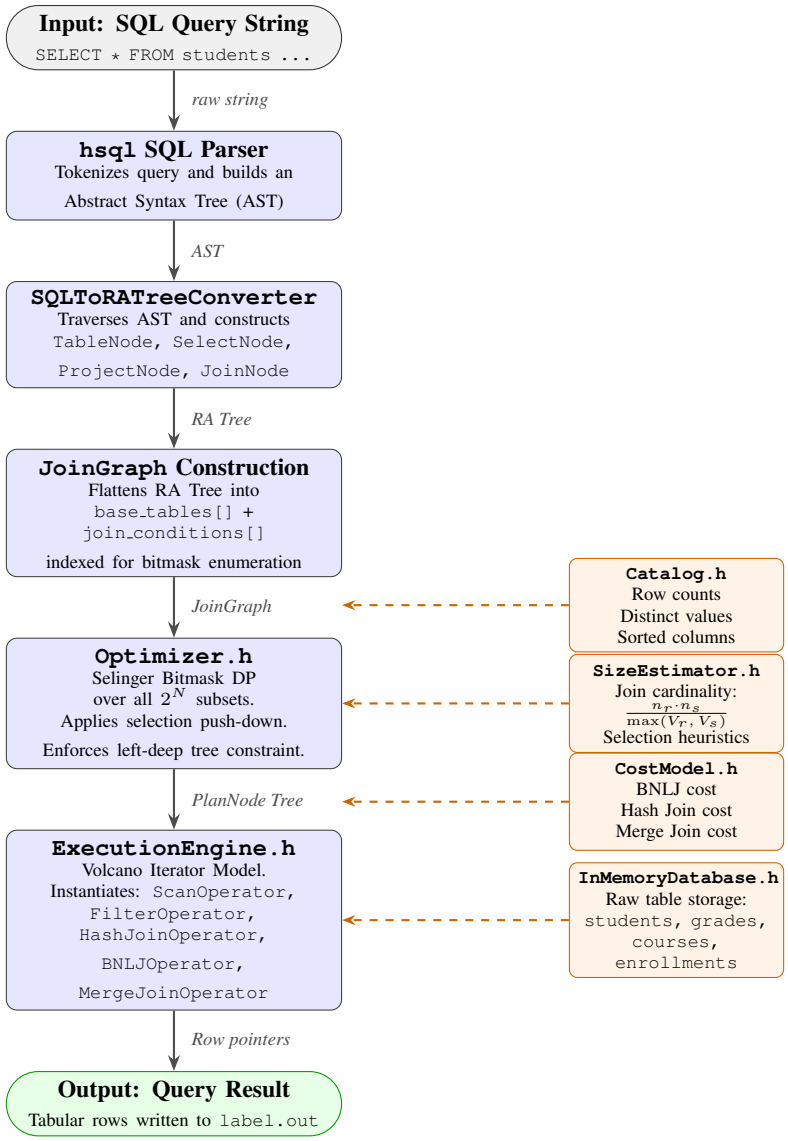

Figure 1: End-to-end query processing pipeline. Solid arrows show the primary data flow from SQL input to tabular output. Dashed arrows show supporting modules that supply metadata and cost estimates to the respective processing stage.

T_b be the average transfer time (0.1 ms) and T_s be the average seek time (10.0 ms). The total cost function is:

$$Cost = (s \times T_s) + (b \times T_b) \tag{1}$$

Because T_s is 100 times larger than T_b, the optimizer is mathematically forced to heavily penalize algorithms that thrash the disk head, heavily favoring sequential scans.

The hardware profile is fixed at $M = 25$ available memory blocks, and tuples are packed at 100 tuples per block. Thus, for any relation R with N_R rows, the block count is $b_R = \lceil N_R/100 \rceil$.

3.2 Block Nested-Loop Join (BNLJ)

BNLJ reads chunks of the outer relation into memory and scans the entire inner relation for each chunk. The algorithm reserves 1 block for the inner relation and reads $M - 1$ blocks of the outer relation.

The number of runs (chunks) required for the outer relation R is:

$$runs = \left\lceil \frac{b_R}{M - 1} \right\rceil \tag{2}$$

The cost functions are formulated as:

- **Transfers:** $b_R + (runs \times b_S)$
- **Seeks:** $2 \times runs$

Optimizer Interaction: The optimizer evaluates both orderings ($R \bowtie S$ and $S \bowtie R$) and retains the cheaper permutation. Because the inner table is re-read once per run, the cost grows linearly with b_R. BNLJ is selected only when the tables are small enough that its total cost is lower than both Hash Join and Sort-Merge Join, which in practice occurs when the tables

fit within a small number of blocks and runs remains close to 1.

3.3 Hash Join (Recursive Partitioning)

For large, unsorted relations, BNLJ becomes impractical as its cost grows quadratically with data size. Hash Join addresses this by partitioning both relations on the join attribute, guaranteeing that any two tuples that could potentially match reside in the same partition. Only tuples within the same partition are ever compared, reducing the effective problem size from N^2 to N.Assuming a uniform hash distribution, the number of partitions is:

$$n = \left\lceil \frac{b_S}{M} \right\rceil \tag{3}$$

The disk I/O requires reading both tables, writing the partitions to disk, and reading them back during the probe phase. The formulas account for partially filled blocks across partitions (the $+n$ overhead):

- **Transfers:** $3(b_R + b_S) + 4n$
- **Reading Seeks:** $k_R = \lceil b_R/M \rceil, k_S = \lceil b_S/M \rceil$
- **Writing Seeks:** $k'_R = \lceil (b_R + n)/M \rceil, k'_S = \lceil (b_S + n)/M \rceil$
- **Total Seeks:** $k_R + k_S + k'_R + k'_S + 2n$

Optimizer Interaction: The optimizer evaluates both build-side configurations symmetrically using R as the build relation and S as the probe relation, and vice versa retaining whichever direction yields the lower cost. Because its total cost scales as $\mathcal{O}(N)$, Hash Join dominates for large unsorted datasets where BNLJ would require hundreds of full inner-table rescans.

3.4 Merge Join (External Sort-Merge)

Sort-Merge Join proceeds in two stages: sorting each input on the join attribute, then performing a single interleaved sequential scan to emit matching pairs. The join phase itself incurs minimal I/O, as both inputs are consumed in one pass. The algorithm's efficiency therefore depends entirely on the cost of the preceding sort.

If the relations are already sorted (verified dynamically by the Optimizer via the `Catalog`), the cost is trivial:

- **Transfers:** $b_R + b_S$
- **Seeks:** $\lceil b_R/M \rceil + \lceil b_S/M \rceil$

When a relation is unsorted, the `CostModel` applies an External Multi-Way Merge Sort. For a relation of b blocks, two sub-cases arise. If $b \leq M$, the entire relation fits in memory; the sort costs $2b$ transfers and 2 seeks (one read pass, one write pass). Otherwise, the initial number of sorted runs produced is $r = \lceil b/M \rceil$, and the number of merge passes required using $M - 1$ input buffers is:

$$P = \lceil \log_{M-1}(r) \rceil \tag{4}$$

The sorting penalty added to the baseline cost is:

- **Sort Transfers:** $2b \times (1 + P)$
- **Sort Seeks:** $2r + (2b \times P)$

Optimizer Interaction: Because $\mathcal{O}(B \log_M B)$ external sorting triggers thousands of random disk seeks, the cost of an unsorted Merge Join usually skyrockets past Hash Join. The Optimizer only selects Merge Join if the Catalog confirms the presence of a pre-sorted column (simulating a Clustered Index).

3.5 Size Estimation and the System-R Fallback

A critical component of cost modeling is cardinality estimation—predicting the number of tuples a join or filter will produce. As outlined in the course lectures, theoretical models rely on the *Uniform Distribution Assumption* to calculate the exact selectivity factor (probability P) for range queries (e.g., $A < x$):

$$P = \frac{x - \min}{\max - \min} \tag{5}$$

However, implementing this continuous mathematical formula presents a significant engineering challenge within our current architecture. The engine's `InMemoryDatabase` stores all records uniformly as type-erased `std::string` objects. This stringification prevents the system from natively distinguishing between a numeric column and a text column. Consequently, performing arithmetic interpolation (calculating $\max - \min$) is impossible without introducing catastrophic casting overhead for every single tuple during the optimization phase. Furthermore, to minimize the memory footprint, the `Catalog` was intentionally designed to track only row counts (N) and distinct values (V), explicitly omitting continuous column boundary statistics like `minValue` and `maxValue`.

To resolve this architectural bottleneck,the `SizeEstimator.h` module bypasses the theoretical uniform distribution formula and instead implements the historical **System-R Fallback Heuristics**. Originally engineered by IBM for exact scenarios where metadata bounds or data types are opaque, these heuristics assign fixed probabilities:

- **Equality ($A = v$):** 10% selectivity (approximating an unknown $1/V$).
- **Aggregate Extrema (MAX/MIN):** 10% selectivity.
- **General Inequalities ($>, <$):** 50% selectivity.

While mathematically blunt compared to exact continuous distributions, these heuristics act as a vital survival mechanism for the optimizer. They guarantee that the Dynamic Programming matrix receives bounded, predictable cardinality estimates, preventing the exponential cost explosions that occur when intermediate join sizes are miscalculated, all without sacrificing the type-agnostic flexibility of the underlying storage layer.

Table 1: Optimizer algorithm selection: computed I/O costs (ms) per test case. Bold entries indicate the algorithm selected by the optimizer.

Test Case	BNLJ	Hash	Merge	Winner
TC9_Tiny 100 × 500 rows	**20.6**	62.2	41.6	BNLJ
TC10_Massive 100K × 500K rows	21,940	**7,456**	251,400	Hash
TC11_Sorted pre-sorted columns	21,940	7,456	**3,000**	Merge

4 Module Breakdown and Interconnectivity

The theoretical mathematics are enforced by a tightly coupled, 10-file C++ architecture. Each module has a single, well-defined responsibility, and communication between modules flows strictly downward through the pipeline.

4.1 Logical Representation Layer

These files bridge the gap between the raw SQL string and the mathematical optimizer, ensuring that no downstream component ever processes unstructured text directly.

4.1.1 `SQLToRATreeConverter.h / .cpp`

Role: Transforms the parsed SQL AST into a logical Relational Algebra (RA) tree.
Mechanics: The module delegates all string-level parsing to the Hyrise `hsql` library, which produces an Abstract Syntax Tree (AST) from the input SQL. The `parseSQLToRA` function then traverses this AST and constructs a typed C++ object tree. Specifically, `FROM` clauses become `TableNode` objects, `WHERE` clauses become `SelectNode` objects wrapping their child, and `SELECT` clauses become `ProjectNode` objects at the root. For a query such as:

```
SELECT * FROM students JOIN grades ON
students.id = grades.student_id WHERE age
> 18
```

the resulting RA tree takes the form:

$$\pi_*\big(\sigma_{\text{age}>18}(\text{students} \bowtie_{\text{id}=\text{student_id}} \text{grades})\big)$$

Interconnectivity: This module acts as a firewall, ensuring that no downstream component ever processes raw SQL strings. It produces a semantically correct `RANode` tree that is used for logical plan display, while separately supplying table and predicate information that feeds into the `JoinGraph` construction.

4.1.2 `JoinGraph.h`

Role: Represents the flattened, index-friendly structure of a query that the DP optimizer can directly enumerate.
Mechanics: The RA tree produced by the converter is hierarchical and recursive, which is well-suited for display but impractical for combinatorial optimization. The `JoinGraph` resolves this by flattening the query into two flat lists: a `base_tables` vector and a `join_conditions` vector of `JoinPredicate` structs of the form $(t_1.c_1 = t_2.c_2)$. For example, a three-table query over {`students`, `grades`, `courses`} produces:

- `base_tables` = [`students`, `grades`, `courses`]
- `join_conditions` = [`students.id = grades.student_id`, `enrollments.course_id = courses.id`]

Interconnectivity: This flat structure is the direct input to `Optimizer.h`. By assigning each table an integer index, the optimizer maps every possible subset of N tables to a unique bitmask in the range $[0,\ 2^N-1]$. For instance, with three tables indexed as bits 0, 1, and 2, the subset {`students`, `grades`} corresponds to the mask $011_2 = 3$, while the full set corresponds to $111_2 = 7$. This representation makes iterating over all 2^N subsets a simple integer loop, which is the foundation of the Selinger DP algorithm.

4.1.3 `PlanNode.h`

Role: The Abstract Syntax Tree (AST) for physical execution.
Mechanics: `PlanNode` structs form a binary tree. Each node contains an `OperatorType` (`SCAN`, `FILTER`, `PROJECT`, `JOIN`) and a physical `JoinAlgorithm` flag (`HASH`, `BNLJ`, `MERGE`).
Interconnectivity: This is the final output of the Optimizer and the input to the `ExecutionEngine.h`. It carries the optimizer's decisions down to the execution factory, forming the bridge between Phase 2 and Phase 3.

4.2 Storage and Metadata Layer

This layer is responsible for data retention and statistical summaries.

4.2.1 `InMemoryDatabase.h`

Role: Acts as the simulated disk storage layer.
Mechanics: The database defines a `Row` as a `std::map<std::string, std::string>` and a `Table` as a `std::vector<Row>`. During initialization, it populates mock tables (e.g., `students`) and massive fact tables (`huge_transactions` with 500,000 rows) used to trigger out-of-core algorithms.
Interconnectivity: It is the lowest layer of the system. Only the `ExecutionEngine` reads its actual contents, while the `Catalog` summarizes it.

4.2.2 `Catalog.h`

Role: The metadata abstraction layer separating the Optimizer from raw data.
Mechanics: The `Catalog` holds a pointer to the `InMemoryDatabase` but restricts interactions to constant-time statistical retrievals: `getRowCount()` and `getDistinct()`. Crucially, it tracks physical schema properties via a `sortedColumns` map.
Interconnectivity: The `Optimizer` and `SizeEstimator` only ever interact with the `Catalog`, never the raw database. The test suite uses `setSortedColumn()` to simulate the dynamic creation of Clustered Indexes at runtime.

4.3 Cost and Statistics Layer

4.3.1 `SizeEstimator.h`

Role: Predicts intermediate relation sizes to inform the Cost Model.
Mechanics: For equijoin operations, `estimateJoinSize` applies the standard uniform-distribution formula:

$$\tilde{n}_{R \bowtie S} = \frac{n_R \times n_S}{\max\big(V(R,A),\ V(S,A)\big)}$$

where $V(R,A)$ and $V(S,A)$ are the distinct value counts retrieved from the `Catalog`. For cross products, where no join predicate exists, the estimate degenerates to $n_R \times n_S$. For selection predicates, the module applies System-R fallback heuristics as described in Section 3.5, returning 10% selectivity for equality and `MAX`/`MIN` conditions, and 50% for general inequalities.
Interconnectivity: The `Optimizer` invokes `SizeEstimator` at every iteration of the DP loop, once for each candidate join pair. The resulting size estimate serves two purposes: it is stored in the `DPState` as the input cardinality for higher-level joins, and it is passed to `CostModel.h` to compute the physical algorithm costs. Inaccurate estimates at any level propagate multiplicatively through the join tree, causing the optimizer to select a suboptimal plan, which makes bounded and consistent estimation essential.

4.3.2 `CostModel.h`

Role: Implements the mathematical models derived in Section 3.
Mechanics: Provides `hashJoinCost()`, `bnljCost()`, and `mergeJoinCost()`, each accepting row counts and returning a simulated I/O cost as a `double`. The primary entry point for the `Optimizer` is `findBestAlgorithm()`, which calls all three functions internally and returns the winning algorithm name alongside its cost by reference. The module is entirely stateless, holding no mutable data.
Interconnectivity: Called once per candidate join pair inside the DP loop. It receives row counts from `SizeEstimator` via the `Optimizer` and returns a scalar cost, functioning purely as a pricing oracle with no side effects.

4.4 The Query Optimizer

4.4.1 `Optimizer.h`

Role: Implements the Selinger-style Dynamic Programming algorithm for cost-optimal join ordering.
Mechanics: The optimizer represents every possible subset of the N base tables as an integer bitmask in $[0,\ 2^N - 1]$. The DP table is filled bottom-up: single-table base cases are initialized first, then subsets of increasing size are built by splitting each bitmask into two non-empty sub-masks and evaluating the cost of joining them. The left-deep constraint is enforced by requiring that one of the two sub-masks always has exactly one bit set, i.e., it represents a single base table. Cross products are penalized and skipped unless no predicate-connected split exists.

Before the DP loop begins, selection predicates from the `JoinGraph` are pushed down to the deepest applicable base case, reducing the effective cardinality used in all subsequent cost calculations.
Each winning configuration is stored in a `DPState` struct containing the cumulative cost, the estimated output size, a pointer to the `PlanNode` tree, and a `sortedOn` string tracking which column the output is ordered on. This last field propagates sortedness information upward, allowing a higher-level join to select Sort-Merge Join without re-paying the sort cost.

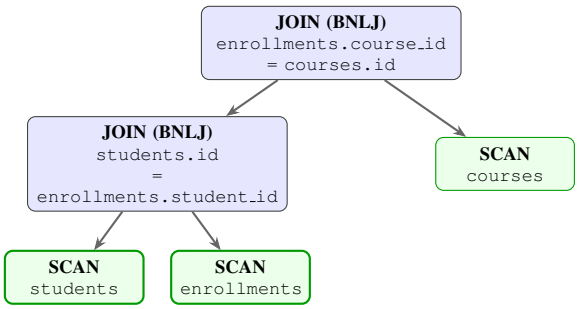

Figure 2: Left-deep `PlanNode` tree produced by the optimizer for TC7 (3-table join). The right child of every join node is always a single base-table scan, satisfying the left-deep constraint.

Interconnectivity: The `Optimizer` is the central orchestrator of the pipeline. It consumes a `JoinGraph`, queries the `Catalog` for sort metadata, calls `SizeEstimator` for intermediate cardinalities, and calls `CostModel` to price each candidate join. Its output is the root `DPState` of the full mask, whose `plan` field is the optimal `PlanNode` tree passed directly to `ExecutionEngine.h`.

4.5 The Physical Execution Engine

4.5.1 `ExecutionEngine.h`

Role: Executes the optimal plan using the Volcano Iterator Model.

Mechanics: The file defines the abstract `Operator` base class with `open()`, `next()`, and `close()` interfaces.

- **`ScanOperator`:** Reads rows directly from `InMemoryDatabase`.
- **`FilterOperator`:** Pipelined operator. Calls `child->next()` until a row matches the predicate.
- **`JoinOperators`:** Implements the physical logic. Hash join acts as a pipeline breaker, reading all left-child tuples into an `std::unordered_map` during its `open()` phase before probing right-child tuples in `next()`.

Interconnectivity: It recursively traverses the `PlanNode` AST to instantiate the physical operator tree. It reads raw data and passes dynamically allocated `Row` pointers up the chain, minimizing the memory footprint to a single tuple per pipeline stage.

5 Testing Framework and Validation

To prove the theoretical correctness of the mathematical models and the dynamic nature of the Optimizer, an automated testing framework was implemented.

5.1 The Test Runner (`run_tests.sh`)

The `run_tests.sh` bash script serves as the entry point for system validation. It invokes the Clang++ compiler with C++17 standards, ensuring all 10 engine headers and the `main.cpp` driver are compiled together.

Crucially, it handles the dynamic linking of the Hyrise SQL Parser (`-Isql-parser/src -Lsql-parser -lsqlparser`). By exporting the `DYLD_LIBRARY_PATH`, it ensures the engine can offload string manipulation to the pre-compiled parser binary before executing the test suite and capturing standard output into `.out` log files.

5.2 The Test Cases (`test.txt`)

The `test.txt` file contains a rigorous suite of queries designed to stress-test specific branches of the Optimizer's DP matrix:

- **TC1–TC4 (Selections):** Validates that the `FilterOperator` correctly pipelines data over simple `ScanOperator`s without triggering join logic.
- **TC5–TC8 (Complex Joins):** Validates heuristic push-downs. The Optimizer must place the `WHERE students.age > 18` predicate as deep in the tree as possible to reduce intermediate join sizes.
- **TC9_Tiny:** Joins `students` (100 rows) and `grades` (500 rows). Because the tables fit entirely within the $M = 25$ blocks, the Cost Model correctly identifies that BNLJ requires 0 extra runs, selecting BNLJ as the optimal physical plan.
- **TC10_Massive:** Joins `huge_users` (100,000 rows) and `huge_transactions` (500,000 rows).

5.3 Dynamic Context Validation (`main.cpp`)

The true power of the engine is proven by the 'main.cpp' test driver, which mutates the 'Catalog' state mid-execution to test the Optimizer's context-awareness.

When 'main.cpp' reads **TC10_Massive**, the 'Catalog' reports that the tables are unsorted. The Optimizer evaluates Merge Join, realizes an External Merge Sort of 6,000 disk blocks will cause catastrophic disk thrashing (Cost $\approx 23,000$), and dynamically pivots to **Hash Join** (Cost $7,456$).

Immediately after, the test runner reads **TC11_Sorted**, which contains the *exact same SQL query*. However, 'main.cpp' intercepts this test case and executes the following logic:

```
if (testId == "TC11_Sorted") {
  catalog.setSortedColumn("huge_users", "id");
  catalog.setSortedColumn("huge_tx", "user_id");
}
```

This simulates the real-time creation of a Clustered Index. The Optimizer re-evaluates the query. During the DP matrix generation, it checks the 'Catalog', detects the new 'sortedOn' property, and passes 'leftSorted = true' to 'CostModel.h'. The Cost Model instantly drops the External Merge Sort penalty. Recognizing that a sequential scan is mathematically optimal, the Optimizer discards Hash Join and selects **Merge Join**, plummeting the query cost to a mere $3,000$ units.

6 Conclusion

This project demonstrates that a fully functional, cost-aware relational query engine can be built from scratch by cleanly separating SQL parsing, mathematical optimization, and physical execution into independent, composable modules. The test suite confirms that the cost model drives genuinely adaptive algorithm selection across varying data scales, from small in-memory joins to multi-hundred-thousand-row scenarios, without any hardcoded rules.

The primary limitation is that the $M = 25$ block constraint governs only cost estimation, not actual runtime memory. Enforcing a real buffer pool in the execution layer would make the system's physical behavior consistent with the model it optimizes for, and represents the most meaningful direction for future work.